

TideWise for LoCO AUV - Integration Guide

Overview

TideWise is now adapted to run on **LoCO AUV** hardware as a coastal flood early-warning system.

This integration leverages:

- **Bar30 Depth Sensor** for atmospheric pressure monitoring
 - **PixHawk IMU** for wave motion detection
 - **Raspberry Pi 4** for edge AI predictions
 - **NOAA GFS data** for regional weather context
 - **ROS** for communication with other LoCO systems
-

Hardware Configuration

Sensors Used from LoCO Parts List

LoCO Component	TideWise Use	Status
Bar30 Depth Sensor	Atmospheric pressure monitoring	☒ In parts list
PixHawk	IMU for wave motion (accelerometer)	☒ In parts list
Raspberry Pi 4	Edge AI + ROS node	☒ In parts list
OLED Screen	Display local alerts	☒ In parts list
USB Cameras (2x)	(Future) Visual tide level detection	☒ In parts list
Ethernet connection	Upload data to cloud	☒ In parts list

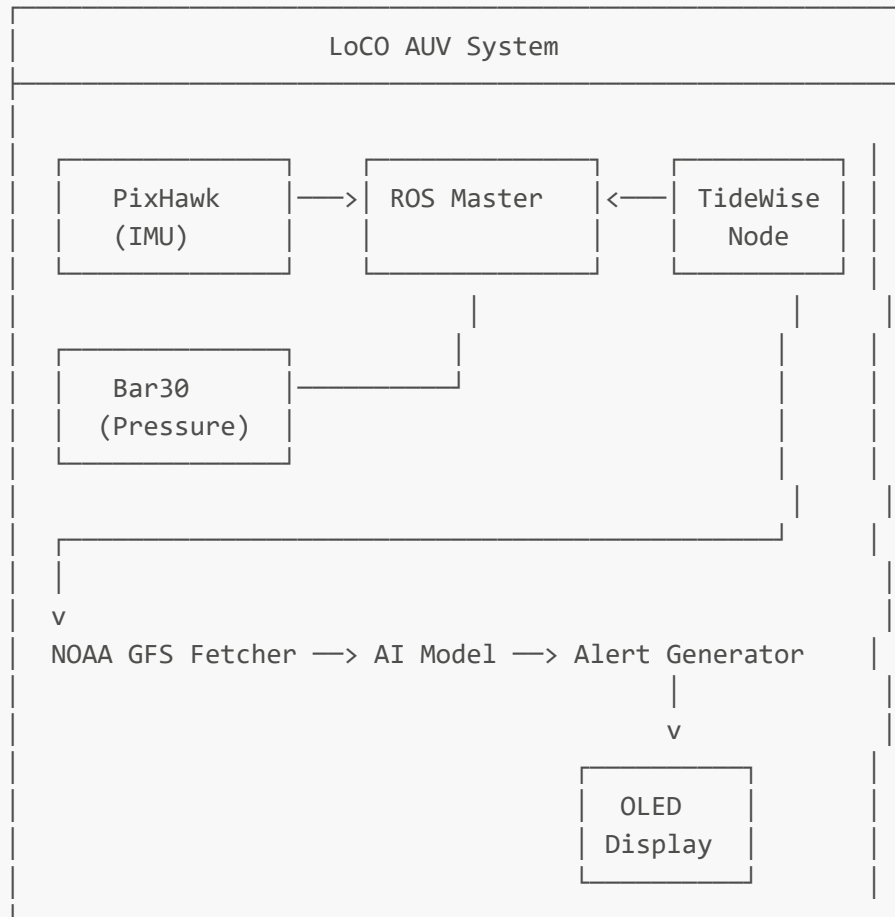
Additional Sensors Needed (Optional Enhancements)

Sensor	Purpose	Cost	Source
DS18B20 Waterproof Temp Sensor	Water temperature (surge indicator)	~R50	Communica, RS Components
JSN-SR04T Ultrasonic Sensor	Tide level measurement	~R150	Communica, Micro Robotics

Note: TideWise can operate with just Bar30 + PixHawk + NOAA data. Additional sensors improve accuracy.

Software Architecture

TideWise as ROS Node



ROS Topics Published by TideWise

/tidewise/alert_level	# String: 'green', 'yellow', 'orange', 'red'
/tidewise/risk_score	# Float: 0.0-10.0
/tidewise/pressure_hpa	# Float: Current atmospheric pressure
/tidewise/wave_height_m	# Float: Estimated wave height
/tidewise/surge_probability	# Float: 0.0-1.0
/tidewise/message	# String: Human-readable alert message

ROS Topics Subscribed by TideWise

/mavros/imu/data	# IMU data from PixHawk (for wave motion)
/bar30/pressure	# Pressure from Bar30 sensor
/bar30/temperature	# Temperature from Bar30

Installation on LoCO Raspberry Pi

Step 1: Prerequisites

```
# SSH into LoCO Raspberry Pi
ssh pi@loco-auv.local

# Update system
sudo apt update && sudo apt upgrade -y

# Install ROS (if not already installed)
# LoCO uses ROS Noetic on Ubuntu 20.04 or ROS Melodic
```

Step 2: Install TideWise Dependencies

```
# Navigate to ROS workspace
cd ~/catkin_ws/src

# Clone TideWise package
git clone https://github.com/YOUR-USERNAME/tidewise_ros.git

# Install Python dependencies
pip3 install numpy pandas requests xarray netCDF4

# Build ROS package
cd ~/catkin_ws
catkin_make
source devel/setup.bash
```

Step 3: Configure TideWise for Your Location

Edit `~/catkin_ws/src/tidewise_ros/config/location.yaml`:

```
location:
  name: "Port Elizabeth"
  latitude: -33.96
  longitude: 25.60
  timezone: "Africa/Johannesburg"

thresholds:
  yellow:
    pressure_drop_3h: 3.0 # hPa
    wave_height: 2.5 # meters
  orange:
    pressure_drop_3h: 5.0
    wave_height: 3.5
  red:
    pressure_drop_3h: 8.0
    wave_height: 5.0
```

Step 4: Launch TideWise Node

```
# Start ROS core (if not already running)
roscore &

# Launch TideWise node
roslaunch tidewise_ros tidewise.launch

# Or run standalone
roslaunch tidewise_ros tidewise_node.py
```

Integration with LoCO Hardware

Reading Bar30 Pressure Sensor

The Bar30 publishes to `/bar30/pressure` (if using BlueRobotics ROS driver).

If not set up yet, we can read directly via I2C:

```
# In tidewise_ros/src/bar30_reader.py
import rospy
from sensor_msgs.msg import FluidPressure
from bar30_python import Bar30 # BlueRobotics Python library

class Bar30Reader:
    def __init__(self):
        self.sensor = Bar30()
        self.pub = rospy.Publisher('/bar30/pressure', FluidPressure,
                                   queue_size=10)

    def read_and_publish(self):
        pressure_mbar = self.sensor.pressure() # Returns pressure in mbar
        pressure_pa = pressure_mbar * 100 # Convert to Pascals for ROS

        msg = FluidPressure()
        msg.header.stamp = rospy.Time.now()
        msg.fluid_pressure = pressure_pa

        self.pub.publish(msg)
```

Reading PixHawk IMU for Wave Motion

PixHawk already publishes IMU data to `/mavros/imu/data`. We extract accelerometer Z-axis:

```
from sensor_msgs.msg import Imu
```

```

class WaveMotionEstimator:
    def __init__(self):
        rospy.Subscriber('/mavros/imu/data', Imu, self.imu_callback)
        self.accel_z_history = []

    def imu_callback(self, msg):
        accel_z = msg.linear_acceleration.z
        self.accel_z_history.append(accel_z)

        # Keep last 10 seconds (assuming 10 Hz IMU)
        if len(self.accel_z_history) > 100:
            self.accel_z_history.pop(0)

    def estimate_wave_height(self):
        """
        Estimate wave height from vertical acceleration variance
        High variance = rough seas
        """
        if len(self.accel_z_history) < 10:
            return 0.0

        variance = np.var(self.accel_z_history)

        # Empirical mapping (needs calibration in real ocean):
        # 0.1 variance ≈ 0.5m waves
        # 1.0 variance ≈ 2.0m waves
        # 5.0 variance ≈ 4.0m waves

        wave_height = 0.3 * np.sqrt(variance)
        return wave_height

```

Displaying Alerts on OLED

The LoCO has an **Adafruit OLED screen** controlled by **Adafruit Trinket Pro**.

We can send serial commands from Raspberry Pi to Trinket to display alerts:

```

import serial

class OLEDDisplay:
    def __init__(self, port='/dev/ttyUSB0'):
        self.serial = serial.Serial(port, 9600)

    def show_alert(self, level, message):
        """
        Send alert to OLED display
        Format: "LEVEL:MESSAGE\n"
        """
        display_text = f"{level.upper()}:{message}\n"

```

```
self.serial.write(display_text.encode())

# Example usage:
display = OLEDDisplay()
display.show_alert('orange', 'FLOOD RISK: Prepare boats')
```

Trinket Pro sketch (Arduino code to upload to Trinket):

```
#include <Wire.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

void setup() {
  Serial.begin(9600);
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(WHITE);
}

void loop() {
  if (Serial.available()) {
    String msg = Serial.readStringUntil('\n');

    display.clearDisplay();
    display.setCursor(0, 0);

    if (msg.startsWith("RED:")) {
      display.setTextSize(2);
      display.println("EMERGENCY");
      display.setTextSize(1);
      display.println(msg.substring(4));
    } else if (msg.startsWith("ORANGE:")) {
      display.println("WARNING");
      display.println(msg.substring(7));
    } else {
      display.println(msg);
    }

    display.display();
  }
}
```

Educational Use: TideWise for Ocean Hub Community

Learning Pathway Integration

TideWise fits into the LoCO learning framework at **Level 3-4**:

Level 3: Python Basics with Ocean Data

- Students fetch NOAA data using TideWise scripts
- Plot pressure trends, wave heights
- Understand storm surge physics

Level 4: ROS Integration

- Students write ROS nodes to read sensors
- Publish custom alerts
- Integrate with LoCO navigation (e.g., "Return to harbor if RED alert")

Hackathon Challenge Integration

October 17-19 Hackathon: "Using sensor data from LoCO, build a business case for coastal communities"

TideWise as Example Solution:

1. **Data Collection:** LoCO deploys near harbor, collects pressure + wave data
 2. **Prediction:** TideWise AI predicts flood risk 6-12 hours ahead
 3. **Business Case:** Municipality pays R40k/year for alerts; saves R2-5M per storm
 4. **Impact:** Community-owned ocean intelligence, local youth employment
-

Next Steps

Immediate (This Week):

1. ☒ Create ROS package structure for TideWise
2. ☒ Write Bar30 sensor reader node
3. ☒ Write wave motion estimator from PixHawk IMU
4. ☐ Test on LoCO hardware (if available)

Short Term (1-3 Weeks):

1. ☐ Add OLED display integration
2. ☐ Create launch files for easy deployment
3. ☐ Document installation for Ocean Hub community
4. ☐ Test with simulated storm data

Medium Term (1-3 Months):

1. ☐ Deploy TideWise on real LoCO in Port Elizabeth harbor
2. ☐ Collect 1 month of real ocean data
3. ☐ Tune alert thresholds based on actual conditions
4. ☐ Create educational modules for schools

Long Term (6-12 Months):

1. ☐ Expand to 5 coastal communities in SA
 2. ☐ Build federated learning network (communities share model improvements)
 3. ☐ Partner with municipalities for subscription revenue
 4. ☐ Submit results to ocean robotics conferences
-

Contact & Collaboration

Want to help build TideWise for LoCO?

- **WhatsApp Group:** [LoCO AUV Mzansi](#)
- **Ocean Hub:** Alexis Grosskopf (alexis@oceanhub.africa)
- **LoCO Community:** David Campey (CoderLevelUp.org)

Questions about TideWise?

- See [tidewise-prototype/README.md](#) for standalone version
 - Check [ros_integration/](#) folder for ROS-specific code
-

License

TideWise is open-source (MIT License) to support the LoCO community's mission:

"communities build robots and robots build communities"

Let's build coastal resilience together!  